

运算符重载

定义

对已有的运算符赋予多重的含义，拓展运算符的适用范围，使之能够作用于对象

形式

- 实质是函数重载
- 可重载为普通函数or成员函数
- 含运算符的表达式——>对运算符函数的调用
- 运算符的操作数——>运算符函数的参数
- 可多次重载，根据类型决定

```
class Complex {
private:
    double real;
    double imag;

public:
    // 构造函数
    Complex(double r = 0, double i = 0) : real(r), imag(i) {}

    // 重载 + 运算符
    Complex operator+(const Complex& other) const {
        return Complex(real + other.real, imag + other.imag);
    }

    // 重载 == 运算符
    bool operator==(const Complex& other) const {
        return (real == other.real && imag == other.imag);
    }

    // 友元函数：重载 << 运算符，用于输出
    friend ostream& operator<<(ostream& out, const Complex& c);
};

// 重载 << 运算符
ostream& operator<<(ostream& out, const Complex& c) {
    out << c.real;
    if (c.imag >= 0)
        out << "+" << c.imag << "i";
}
```

```

    else
        out << c.imag << "i";
    return out;
}

int main() {
    Complex c1(3, 4);
    Complex c2(1, 2);

    Complex c3 = c1 + c2;

    cout << "c1 = " << c1 << endl;
    cout << "c2 = " << c2 << endl;
    cout << "c1 + c2 = " << c3 << endl;

    if (c1 == c2)
        cout << "c1 和 c2 相等" << endl;
    else
        cout << "c1 和 c2 不相等" << endl;

    return 0;
}

```

不能重载的运算符

- `.` 成员访问运算符
- `.*` 成员指针访问运算符
- `::` 作用域解析运算符
- `?:` 三目运算符
- `sizeof`

只能重载为成员函数的运算符

以下 **4个运算符** 在C++中只能重载为类的成员函数，不能重载为全局（友元）函数：

运算符	名称
<code>=</code>	赋值运算符
<code>[]</code>	下标运算符
<code>()</code>	函数调用运算符
<code>-></code>	成员访问运算符

赋值运算符 = 的重载

1. 只能重载为成员函数
2. 注意深拷贝与浅拷贝
3. 返回值类型最好为 & （保留连等的特性）

++运算符的重载

在 C++ 里，前置和后置 ++ 的重载是靠**函数参数列表**区分的：

```
// 前置 ++x
CHugeInt& operator++();

// 后置 x++
CHugeInt operator++(int);
```

关键点：

1. 前置：operator++()
 - 没有参数
 - 通常返回引用 T& （返回加完后的自身）
2. 后置：operator++(int)
 - 多出来的 int 是“占位参数”，不使用，只为和前置区分
 - 通常返回值 T （返回加之前的旧值副本）

Rule of Three

类型转换运算符的重载

```
#include <iostream>
using namespace std;

class Number {
private:
    double value;

public:
    Number(double v) : value(v) {}

    // 类型转换运算符重载
    operator int() {
```

```

        return (int)value;
    }
};

int main() {
    Number n(3.14);

    int x = n;    // 自动调用 operator int()
    cout << "x = " << x << endl;

    return 0;
}

```

注意事项：

被重载时不能写返回值类型

自增自减运算符的重载

```

#include <iostream>
using namespace std;

class Number {
private:
    int value;

public:
    Number(int v) : value(v) {}

    // 前置++
    Number& operator++() {
        ++value;
        return *this;
    }

    // 后置++
    Number operator++(int) {
        Number temp = *this;
        value++;
        return temp;
    }

    void show() {
        cout << "value = " << value << endl;
    }
}

```

```
    }  
};  
  
int main() {  
    Number n(5);  
  
    ++n;      // 前置++  
    n.show(); // 6  
  
    n++;      // 后置++  
    n.show(); // 7  
  
    return 0;  
}
```

- 前置函数作为**一元**运算符重载，后置函数作为**二元**运算符重载，多写一个没用的 int 参数
- 前置函数多返回**引用**，后置函数多返回**值**